

# TEMA 3

## TÉCNICAS PARA OPTIMIZAR EL USO DEL HW



# TEMA 3

## TÉCNICAS PARA OPTIMIZAR EL USO DEL HW

### ARQUITECTURA DE COMPUTADORES

*Dpto. Arquitectura de Computadores*

*Universidad de Málaga*



# CONTENIDOS

- Introducción
- Loop reordering (interchange)
- Buffering
- Loop unrolling (desenrollado de bucle)
- Blocking

Transparencias basadas en el curso:

- Oscar Plata. Altas prestaciones para datos y ciberseguridad. Universidad de Málaga
- Sección *Going Faster* del libro D. A. Patterson and J. L. Hennessy, Computer Organization and Design, 5th ed. Morgan Kaufmann, 2013



# INTRODUCCIÓN

TEMA 3: TÉCNICAS PARA OPTIMIZAR EL USO DEL HW



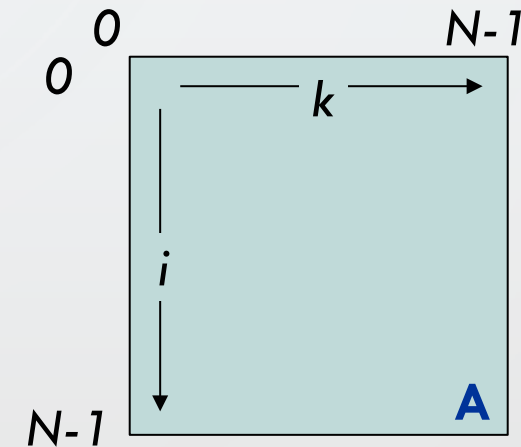
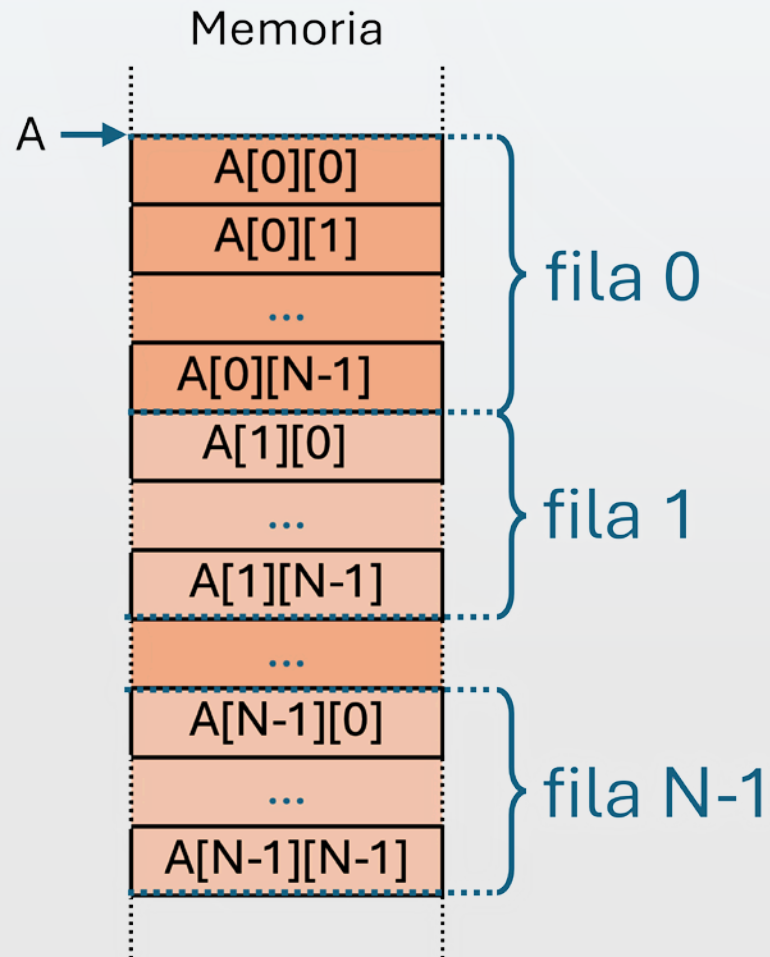
# INTRODUCCIÓN

- El objetivo de esta práctica es conocer algunas técnicas generales de optimización para mejorar el rendimiento del código serie
  - Técnicas relacionadas con el acceso a la jerarquía caché del sistema
- Utilizaremos un código que calcula el **producto de dos matrices** cuadradas:
  - Aparece en numerosas aplicaciones de diferentes ámbitos (aplicaciones gráficas, de simulación, científicas, etc.)
  - posee una estructura fácilmente extrapolable a otros códigos
- Usaremos **lenguaje de programación C**
  - Ejecutaremos sobre un sistema Linux



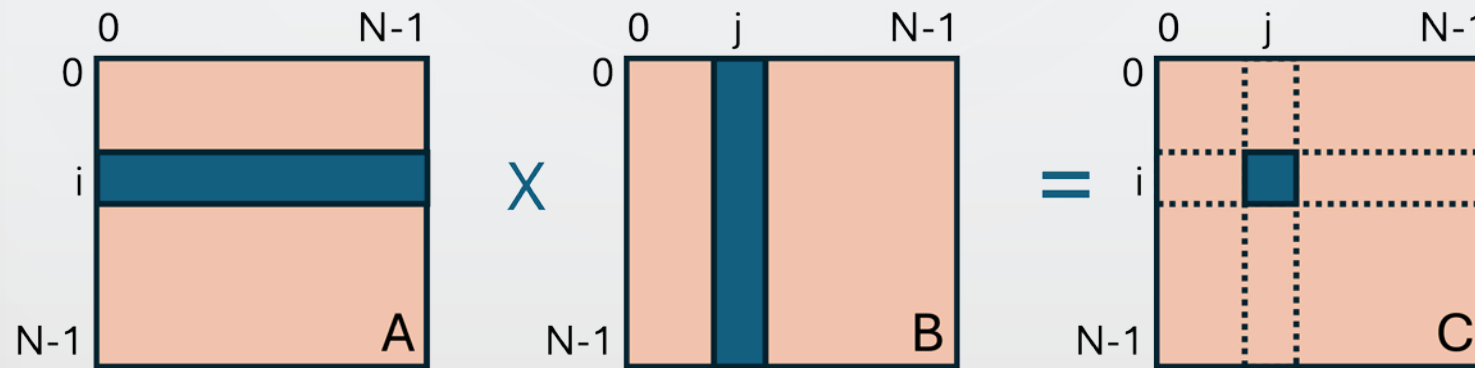
# MATRICES EN C

- Cómo organizan los compiladores C/C++ las matrices 2D en memoria



# MULTIPLICACIÓN DE MATRICES

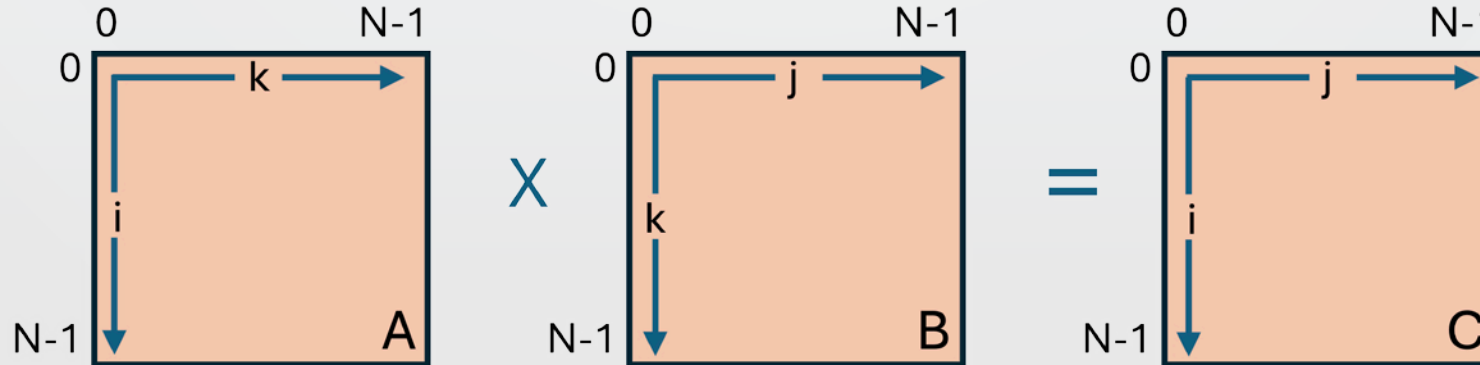
- El producto de dos matrices,  $A$  y  $B$ , de tamaño  $N \times N$  da como resultado otra matriz,  $C$ , de tamaño  $N \times N$  donde el elemento  $C[i][j]$  vendrá dado por el producto escalar de la fila  $i$ -ésima de  $A$  por la columna  $j$ -ésima de  $B$



# MULTIPLICACIÓN DE MATRICES

- Recorrido de las iteraciones suponiendo el orden de los bucles:  $j, k, i$

```
// Assuming A[N][N], B[N][N], C[N][N];  
  
for (j = 0; j < N; j++)  
  for (k = 0; k < N; k++)  
    for (i = 0; i < N; i++)  
      C[i][j] += A[i][k] * B[k][j];
```

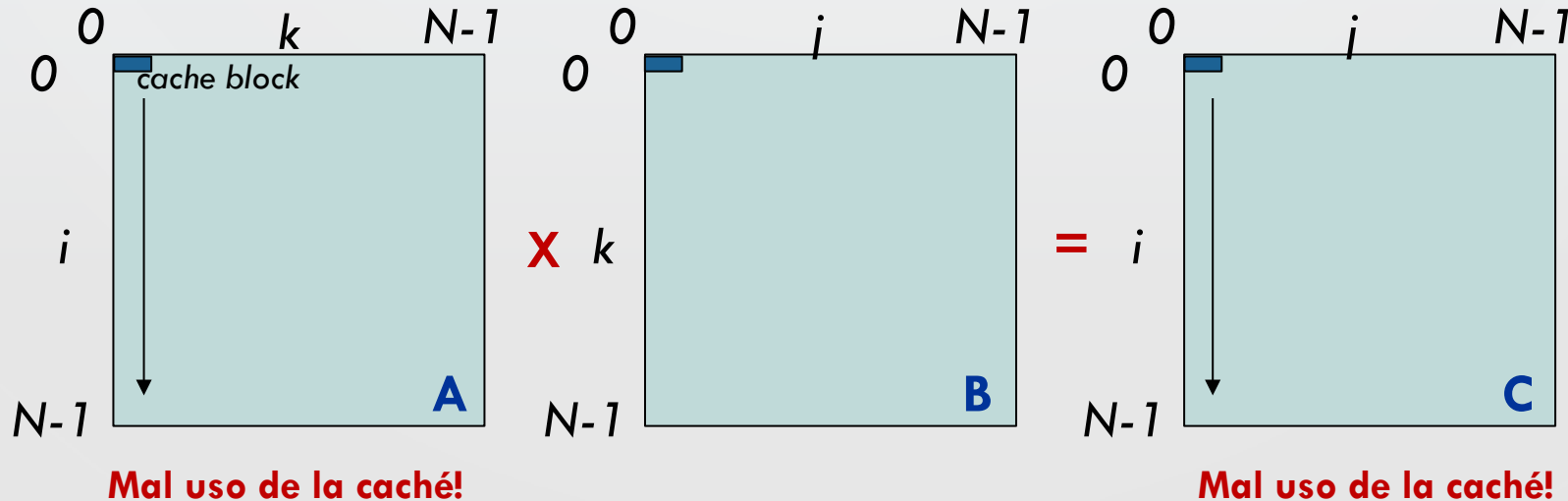




# MULTIPLICACIÓN DE MATRICES

- Análisis del comportamiento de la caché, suponiendo el orden de los bucles:  $j, k, i$

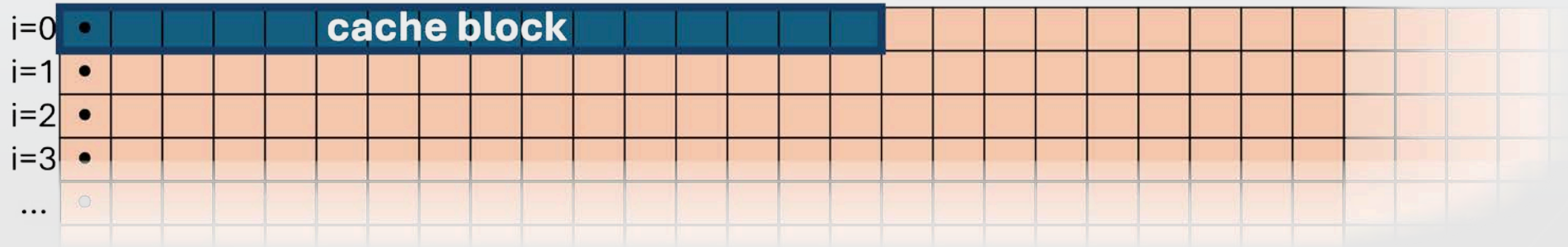
```
// Assuming A[N][N], B[N][N], C[N][N];  
  
for (j = 0; j < N; j++)  
  for (k = 0; k < N; k++)  
    for (i = 0; i < N; i++)  
      C[i][j] += A[i][k] * B[k][j];
```



# MULTIPLICACIÓN DE MATRICES

- Análisis del comportamiento de la caché, suponiendo el orden de los bucles:  $j, k, i$

```
// Assuming A[N][N], B[N][N], C[N][N];  
  
for (j = 0; j < N; j++)  
  for (k = 0; k < N; k++)  
    for (i = 0; i < N; i++)  
      C[i][j] += A[i][k] * B[k][j];
```



# LOOP REORDERING (INTERCHANGE)

TEMA 3: TÉCNICAS PARA OPTIMIZAR EL USO DEL HW

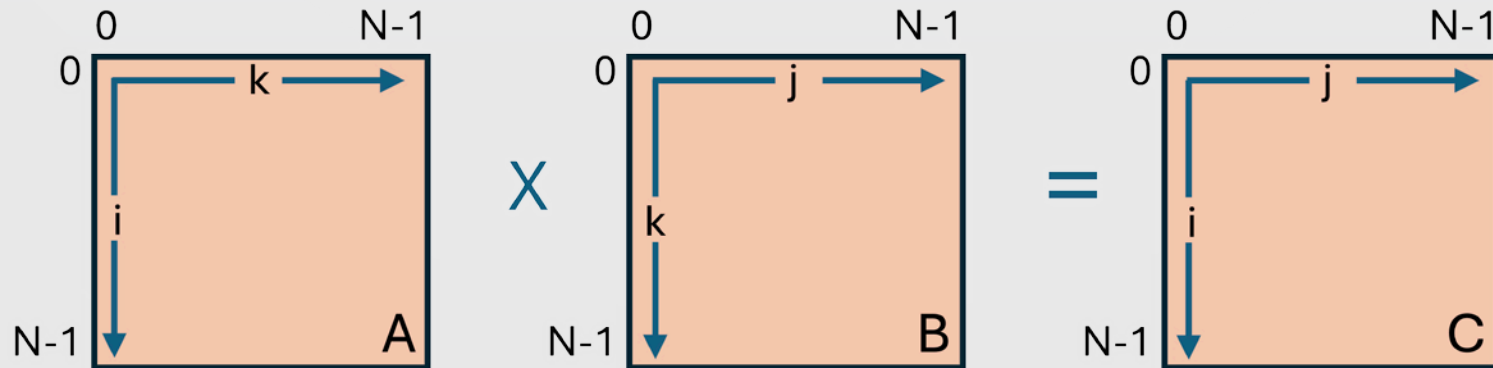


# LOOP REORDERING (INTERCHANGE)

- Esta técnica consiste en el intercambio de los bucles para hacer que el acceso a los elementos de las matrices se realice de forma que se maximice el acceso a los elementos que se traen en un bloque de caché

○ Permite reducir la tasa de fallos de caché

```
// Assuming A[N][N], B[N][N], C[N][N];  
  
for (i = 0; i < N; i++)  
  for (k = 0; k < N; k++)  
    for (j = 0; j < N; j++)  
      C[i][j] += A[i][k] * B[k][j];
```



# **BUFFERING**

**TEMA 3: TÉCNICAS PARA OPTIMIZAR EL USO DEL HW**



# BUFFERING

- Esta técnica consiste en precargar en un registro del procesador los elementos de las matrices que en el bucle más interno son leídos/escritos en cada iteración del bucle (ya que no cambian sus índices)
  - Permite reducir las referencias de memoria a la caché de datos
  - Permite simplificar la aritmética de indexación de matrices

```
// Assuming A[N][N], B[N][N], C[N][N];  
  
for (i = 0; i < N; i++)  
    for (k = 0; k < N; k++)  
        register float aik = A[i][k];  
        for (j = 0; j < N; j++)  
            C[i][j] += aik * B[k][j];
```



# LOOP UNROLLING (DESEENROLLADO DE BUCLE)

TEMA 3: TÉCNICAS PARA OPTIMIZAR EL USO DEL HW



# LOOP UNROLLING (DESENROLLADO DE BUCLE)

- Esta técnica consiste en hacer varias copias del código dentro del bucle (tantas como factor de desenrollado se quiera aplicar) y utilizando registros distintos para cada una de las copias (renombrado de registros)
  - Permite aumentar el tamaño del cuerpo del bucle (número de instrucciones), separando las instrucciones de acceso a memoria de las instrucciones que usan esos datos
  - Permite reducir el número de bucles/saltos (instrucciones de control)

```
// Assuming A[N][N], B[N][N], C[N][N];  
for (i = 0; i < N; i++)  
    for (k = 0; k < N; k++)  
        for (j = 0; j < N; j+=2)  
        {  
            C[i][j] += A[i][k] * B[k][j];  
            C[i][j+1] += A[i][k] * B[k][j+1];  
        }
```

Desenrollado en un factor  
de 2 del bucle  $j$





# BLOCKING

TEMA 3: TÉCNICAS PARA OPTIMIZAR EL USO DEL HW



# BLOCKING

- A veces, el uso de técnicas como el *loop interchange* para optimizar la localidad espacial de la caché es suficiente para obtener el máximo beneficio de la jerarquía de la caché
  - Sin embargo, en muchos casos, es mejor explotar tanto la **localidad espacial** como la **temporal**
- Esta técnica divide las matrices en submatrices y se realizan productos parciales con ellas acumulando los resultados en submatrices para obtener el resultado final
  - Permite explotar tanto la **localidad espacial** como la **temporal**

$$C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix} \quad A = \begin{bmatrix} A_1 & A_2 \\ A_3 & A_4 \end{bmatrix} \quad B = \begin{bmatrix} B_1 & B_2 \\ B_3 & B_4 \end{bmatrix}$$



# BLOCKING

## Blocking de 2 bucles anidados

```
// Bucle anidado
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++){
    A[i][j] = ...
  }
```

*División iteraciones*

```
#define NB ??

// 2 bucles anidados(blocked)
for (i0 = 0; i0 < N; i0 += NB)
  for (i = i0; i < min(N,i0+NB); i++)
    for (j0 = 0; j0 < N; j0 += NB)
      for (j = j0; j < min(N,j0+NB); j++){
        A[i][j] = ...
      }
```

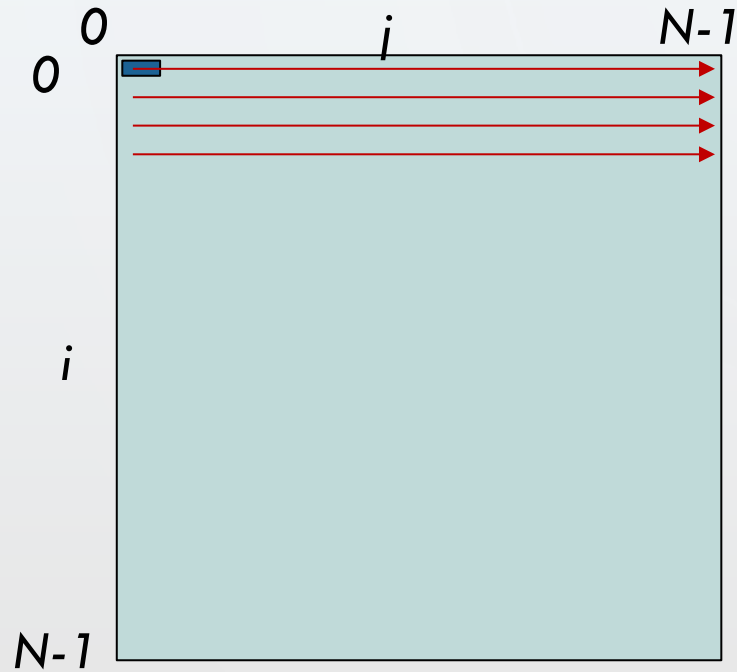
*interchange*

```
#define NB ??

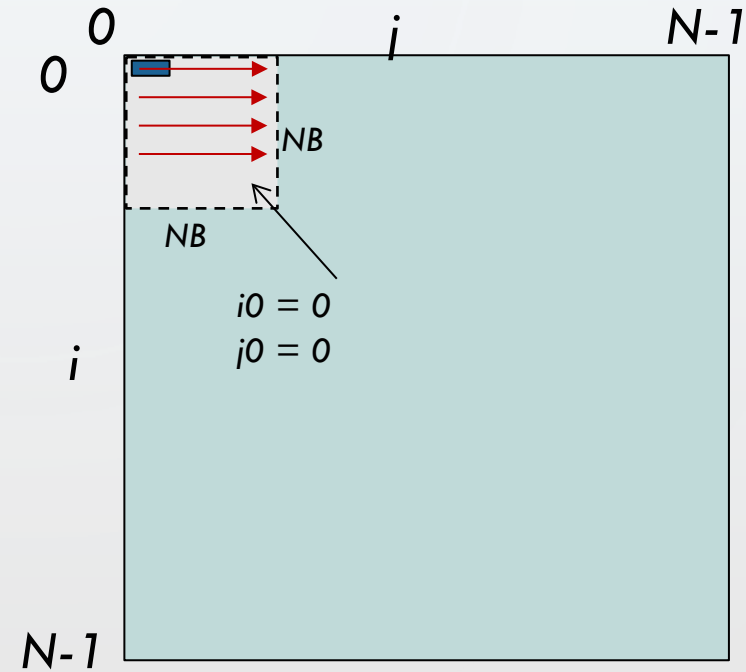
// 2 bucles anidados(blocked)
for (i0 = 0; i0 < N; i0 += NB)
  for (j0 = 0; j0 < N; j0 += NB)
    for (i = i0; i < min(N,i0+NB); i++)
      for (j = j0; j < min(N,j0+NB); j++){
        A[i][j] = ...
      }
```

# BLOCKING

Blocking de 2 bucles anidados



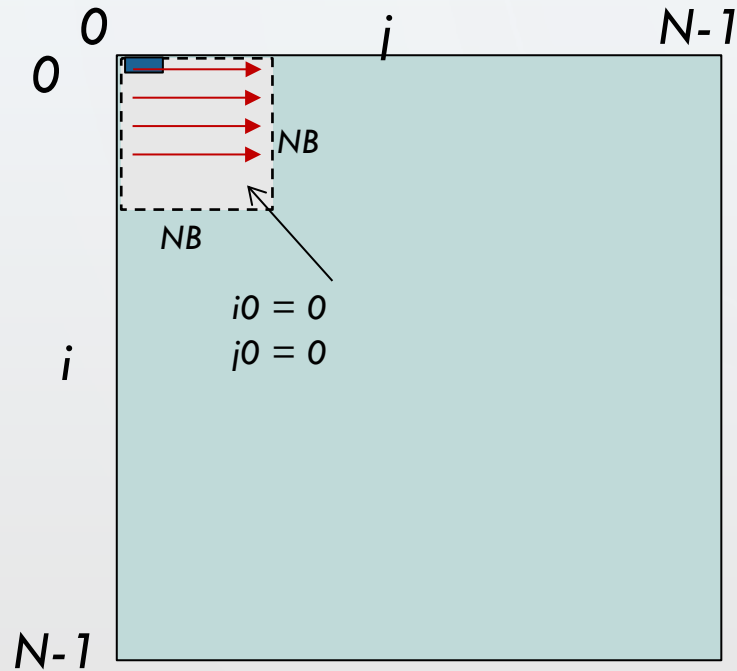
```
for (i = 0; i < N; i++)  
  for (j = 0; j < N; j++){  
    A[i][j] = ...  
  }
```



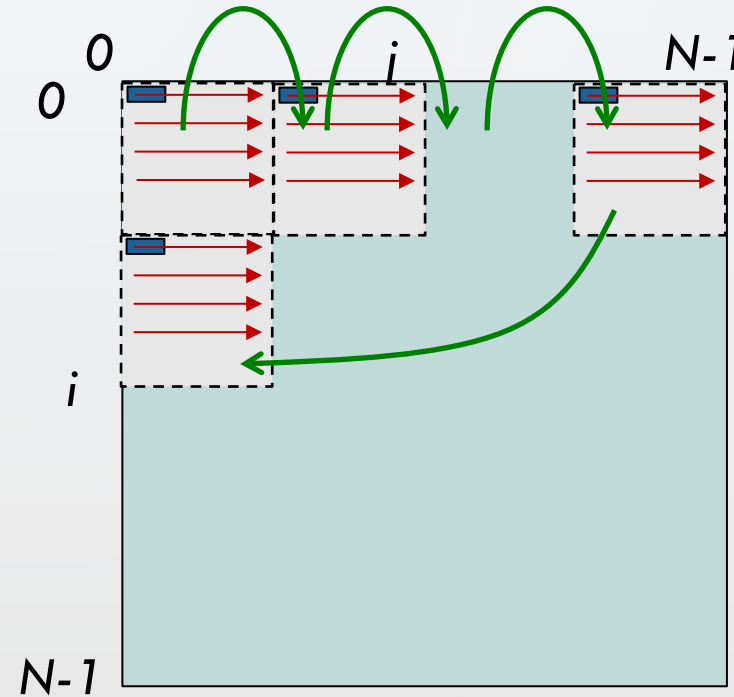
```
for (i0 = 0; i0 < N; i0 += NB)  
  for (j0 = 0; j0 < N; j0 += NB)  
    for (i = i0; i < min(N, i0+NB); i++)  
      for (j = j0; j < min(N, j0+NB); j++){  
        A[i][j] = ...  
      }
```

# BLOCKING

Blocking de 2 bucles anidados



```
for (i0 = 0; i0 < N; i0 += NB)
  for (j0 = 0; j0 < N; j0 += NB)
    for (i = i0; i < min(N, i0+NB); i++)
      for (j = j0; j < min(N, j0+NB); j++){
        A[i][j] = ...
      }
}
```



```
for (i0 = 0; i0 < N; i0 += NB)
  for (j0 = 0; j0 < N; j0 += NB)
    for (i = i0; i < min(N, i0+NB); i++)
      for (j = j0; j < min(N, j0+NB); j++){
        A[i][j] = ...
      }
}
```